

1. Selection Problem with divide & conquer approach.Introduction:-

The selection problem is an important problem in computer science that involves finding the k^{th} smallest element in an unsorted list. It is widely used in statistics, data analysis & algorithm design.

Objective:-

To implement a program that finds the k^{th} smallest element in an array using the divide & conquer approach.

Theory:-

The selection problem can be solved using divide & conquer technique similar to Quick sort. The algorithm partitions the array into two parts based on a pivot element. If the pivot position matches the required position, the element is found. Otherwise, the algorithm continues searching in the left or right subarray.

Algorithm:-

1. Read the no. of elements in the array.
2. Read the array elements.
3. Choose a pivot element.
4. Partition the array into left & right parts.
5. If pivot position = k , return the element.
6. If $k <$ pivot position, repeat in left subarray.
7. Otherwise, repeat in right subarray.
8. Display the k^{th} smallest element.

Code:-

```
int partition (int arr[], int low, int high)
```

```
{ int pivot = arr[high];
```

```
  int i = low - 1;
```

```
  for (int j = low; j < high; j++)
```

```
  { if (arr[j] <= pivot)
```

```
    { i++;
```

```
      swap(arr[i], arr[j]);
```

```
    }
```

```
  }
```

```

swap arr[i+1], arr[high]);
return i+1;
}
int kthSmallest(int arr[], int low, int high, int k)
{
    if (low <= high)
    {
        int pos = partition(arr, low, high);
        if (pos == k-1)
            return arr[pos];
        else if (pos > k-1)
            return kthSmallest(arr, low, pos-1, k);
        else
            return kthSmallest(arr, pos+1, high, k);
    }
    return -1;
}

int main()
{
    int n, k;
    cout << "Enter no. of elements : ";
    cin >> n;
    int arr[n];
    cout << "Enter elements : ";
    for (int i = 0; i < n; i++)
        cin >> arr[i];
    cout << "Enter k : ";
    cin >> k;
    cout << "k-th smallest element is : " << kthSmallest(arr, 0, n-1, k);
    return 0;
}

```

Output:-

```

Enter no. of element : 6
Enter elements : 7 10 4 3 20 15
Enter k : 3
kth smallest element is : 7

```

Time complexity :-

- Best case : $O(n)$
- Average case : $O(n)$
- Worst case : $O(n^2)$

Conclusion :-

The greedy algorithm were successfully implemented. These algorithms efficiently solve optimization and graph problems. using locally optimal choices.

References :-

Introduction to Algorithm by Cormen

Design and Analysis of Algorithms by Anany Levitin.

2. Fractional Knapsack problem, Job sequencing with deadline, Kruskal algorithm, Prim algorithm, Dijkstra's algorithm.

Introduction :-

Greedy algorithms are used to solve optimization problems where the best choice is made at each step to achieve the optimal solution. These algorithms are efficient and widely used in scheduling, network design and resource allocation.

Objective :-

To implement the following greedy algorithms:

- Fractional Knapsack
- Job sequencing with deadline
- Prim's Algorithm
- Dijkstra's Algorithm

Theory :-

Greedy algorithms work by selecting the best possible option at every step. These algorithms do not reconsider previous decision. Fractional Knapsack selects items based on profit-to-weight ratio, Job sequencing selects jobs based on profit, Kruskal's & Prim's algorithm are used to find minimum spanning trees, and Dijkstra's algorithm is used to find the shortest path in a graph.

a. Fractional knapsack:

```
int main() {
    int n, w;
    float p[50], w[50], r[50], map = 0;
    cin >> n;
    for (int i = 0; i < n; i++) {
        cin >> p[i] >> w[i];
        r[i] = p[i] / w[i];
    }
    for (int i = 0; i < n; i++)
        for (int j = i + 1; j < n; j++)
            if (r[i] < r[j]) {
                swap(r[i], r[j]);
                swap(p[i], p[j]);
                swap(w[i], w[j]);
            }
    cin >> w;
```

```

for (int i = 0; i < n; i++) {
    if (w >= w[i]) {
        max p += p[i];
        w = w[i];
    } else {
        max p = r[i] * w;
        break;
    }
}
cout << "max profit = " << max p;
}

```

b) Job sequencing with Deadline:

```

int main() {
    int n, d[50], p[50], slot[50] = {0};
    cin >> n;
    for (int i = 0; i < n; i++)
        cin >> d[i] >> p[i];
    for (int i = 0; i < n; i++)
        for (int j = i + 1; j < n; j++)
            if (p[i] < p[j]) {
                swap(p[i], p[j]);
                swap(d[i], d[j]);
            }
    int profit = 0;
    for (int i = 0; i < n; i++)
        for (int j = d[i] - 1; j >= 0; j--)
            if (slot[j] == 0) {
                slot[j] = 1;
                profit += p[i];
                break;
            }
    cout << "max profit = " << profit;
}

```

c) Kruskal's Algorithm:

```

int parent[50];
int find(int i){
    while (parent[i] != parent[i]);
    return i;
}

int main(){
    int n, e, u[50], v[50], w[50];
    cin >> n >> e;
    for (int i = 0; i < e; i++) cin >> u[i] >> v[i] >> w[i];
    for (int i = 0; i < e; i++)
        for (int j = i + 1; j < e; j++)
            if (w[i] > w[j]){
                swap(w[i], w[j]);
                swap(u[i], u[j]);
                swap(v[i], v[j]);
            }
    for (int i = 0; i < e; i++){
        int a = find(u[i]);
        int b = find(v[i]);
        if (a != b){
            cout << u[i] << " - " << v[i] << endl;
            parent[b] = a;
        }
    }
}

```

d) Prim's Algorithm:

```

int main(){
    int n, c[100][100], v[100] = {0};
    cin >> n;
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            cin >> c[i][j];
    v[0] = 1;
    for (int k = 0; k < n - 1; k++){
        int min = 999, m, y;
        for (int i = 0; i < n; i++)
            for (int j = 0; j < n; j++)
                if ((c[i][j] < 6 & v[i] < 6 & v[j] < 6 & c[i][j] < min) &

```



```

    min = a[i][j];
    x = i; y = j;

    cout << " " << y << endl;
    v[j] = 1;
}
}

```

Ex) Dijkstra's Algorithm:

```

int main() {
    int n, a[10][10], d[10], v[10] = {0};
    cin >> n;
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            cin >> a[i][j];
    for (int i = 0; i < n; i++) d[i] = a[0][i];
    v[0] = 1;
    for (int i = 0; i < n; i++) {
        int min = 999, u;
        for (int j = 0; j < n; j++)
            if (!v[j] && d[j] < min) {
                min = d[j];
                u = j;
            }
        v[u] = 1;
        for (int j = 0; j < n; j++)
            if (!v[j] && d[u] + a[u][j] < d[j])
                d[j] = d[u] + a[u][j];
    }
    for (int i = 0; i < n; i++) cout << d[i] << " ";
}

```

Time complexity :-

- Fractional Knapsack : $O(n \log n)$
- Job sequencing with Deadline : $O(n^2)$
- Kruskal's Algorithm : $O(E \log E)$
- Prim's Algorithm : $O(V^2)$
- Dijkstra's Algorithm : $O(V^2)$

Output :-

a. Fractional Knapsack :-

Enter no. of items : 3

Enter profit & weight :

60 10

100 20

120 30

Enter capacity : 50

max profit = 240

b. Job-sequencing with Deadline :-

Enter no. of jobs : 5

Enter deadline & profit :

2 100

1 19

1 25

3 15

max profit = 142

c. Kruskal's Algorithm :-

Enter no. of vertices & edges : 4 5

Enter edges (u v w) :

0 1 10

0 2 6

0 3 5

1 3 15

2 3 4

Edges in MST :

2 - 3

0 - 3

0 - 1

d. Prim's Algorithm :-

Enter no. of vertices : 4

Enter cost matrix :

0 10 6 5

10 0 0 15

6 0 0 4

5 15 4 0

Edges in MST :

0 - 3

3 - 2

0 - 1

e. Dijkstra's Algorithm :

Enter no. of vertices : 4

Enter cost matrix :

0 10 3 0

0 0 1 2

0 4 0 8

0 0 0 0

Shortest distances from source (0) :

Conclusion :-

The greedy algorithms were successfully implemented. These algorithms efficiently solve optimization & graph problems using locally optimal choices.

References :-

Introduction to Algorithms by Cormen
Design & Analysis by Anany Levitin.

3.1 Implement the dynamic programming algorithms.

Introduction:-

Dynamic programming is an algorithm design technique used to solve complex problems by breaking them into smaller sub-problems and storing their results to avoid repeated calculations.

Objective:-

To implement the following Dynamic Programming algorithms:

- Fibonacci using Dynamic Programming
- 0/1 Knapsack using Dynamic Programming
- Longest common subsequence
- matrix chain multiplication

Theory:-

Dynamic programming works on two important principles:

1. Overlapping subproblems - The same problem is solved multiple times.
2. Optimal substructure - The optimal solution of problem can be constructed from optimal solution of its subproblems.

a. Fibonacci using Dynamic programming

Algorithm:

1. Read the no. of terms
2. Store the first two terms as 0 and 1.
3. Use a loop to compute the next terms.
4. ~~Disp~~ store results in an array.
5. Display the fibonacci series.

code:

```
int main()
{
    int n, f[50];
    cout << "enter no. of terms: ";
    cin >> n;
    f[0] = 0;
    f[1] = 1;
    for (int i = 2; i < n; i++)
        f[i] = f[i-1] + f[i-2];
    for (int i = 0; i < n; i++)
        cout << f[i] << " ";
    return 0;
}
```

b. 0/1 Knapsack using Dynamic programming

Algorithm:

1. Read no. of items
2. Read profit & weight
3. Create a table $dp[n+1][w+1]$
4. Fill the table using dynamic programming.
5. Display the max profit.

Code:-

```
int main() {
    int n, w, p[50], wt[50], dp[50][50];
    cout << "enter no. of items: ";
    cin >> n;
    cout << "enter profit & weight: \n";
    for (int i = 1; i <= n; i++)
        cin >> p[i] >> wt[i];
    cout << "enter capacity: ";
    cin >> w;
    for (int i = 0; i <= n; i++)
        for (int j = 0; j <= w; j++)
            if (i == 0 || j == 0)
                dp[i][j] = 0;
            else if (wt[i] <= j)
                dp[i][j] = max(p[i] + dp[i-1][j-wt[i]], dp[i-1][j]);
            else
                dp[i][j] = dp[i-1][j];
    cout << "max profit = " << dp[n][w];
}
```

c. Longest common subsequence

Algorithm:

1. Read two strings
2. Create a table $dp[m+1][n+1]$
3. Compare characters
4. Fill the table using DP.
5. Display length of LCS.

code:

```
int main()
{
    char a[50], b[50];
    int dp[50][50];
    cout << "Enter first string: ";
    cin >> a;
    cout << "enter second string: ";
    cin >> b;
    int m = strlen(a);
    int n = strlen(b);
    for (int i = 0; i <= m; i++)
        for (int j = 0; j <= n; j++)
            if (i == 0 || j == 0)
                dp[i][j] = 0;
            else if (a[i-1] == b[j-1])
                dp[i][j] = dp[i-1][j-1] + 1;
            else
                dp[i][j] = max(dp[i-1][j], dp[i][j-1]);
    cout << "Length of LCS = " << dp[m][n];
}
```

d. matrix chain multiplication:

Algorithm:-

1. Read no. of matrices
2. Read dimensions
3. Create Dp table
4. compute min multiplicative cost
5. Display result

code:-

```
int main()
{
    int n, p[50], m[50][50];
    cout << "enter no. of matrices: ";
    cin >> n;
    cout << "enter dimensions: ln";
    for (int i = 0; i <= n; i++)
        cin >> p[i];
    for (int i = 1; i <= n; i++)
        m[i][i] = 0;
}
```



```

for (int l=2; l<=n; l++)
    for (int i=1; i<=n-l+1; i++) {
        int j = i+l-1;
        m[i][j] = 99999;
        for (int k=i; k<j; k++)
            m[i][j] = min (m[i][k],
                m[i][k] + m[k+1][j] + p[i-1]*p[k]*p[j]);
    }
cout << "min multiplications = " << m[1][n];

```

Output :-

a) fibonacci using Dynamic programming

Time complexity: $O(n)$

Input: Enter no. of terms = 7

Output: fibonacci Series: 0 1 1 2 3 5 8

b) 0/1 Knapsack using Dynamic programming

Time complexity: $O(nw)$

Input:

Enter no. of items: 3

Enter profit & weight:

60 10

100 20

120 30

Enter capacity: 50

Output: maximum profit = 220

~~Length of LCS = 3~~

c) Longest common subsequence:

Time complexity: $O(mn)$

Input:

Enter first string: ABCDGH

Enter second string: AEDFHR

Output: Length of LCS = 3

d) matrix chain multiplication:

Time complexity: $O(n^3)$

Input:

Enter no. of matrices: 4

Enter dimensions:

10 20 30 40 30

Output :
minimum multiplications = 30000

Conclusion :-

The dynamic programming algorithms were successfully implemented. The programs reduced time complexity by storing intermediate results & avoided repeated computations.

Reference :-

Introduction to Algorithm - Thomas H. Cormen

Design & Analysis of Algorithms - Anany Levitin